Modules: import and export in TypeScript

In TypeScript, a **module** is a file that contains code such as variables, functions, classes, or interfaces that can be **shared with other files**.

Modules help us divide a large program into **small, reusable files**.

`export` → makes code available outside a file.
`import` → brings exported code into another file.

1. Why use Modules?

Suppose we have a large student management program:

```
Student Project
│
├── student.ts
├── calculator.ts
└── main.ts
```

Instead of writing everything in one file, we can keep related code in separate files.

```
student.ts      → Student class
calculator.ts   → Calculator functions
main.ts         → Uses Student and Calculator
```

Example 1: Exporting a Function

File: `calculator.ts`

```
export function add(a: number, b: number): number {
    return a + b;
}

export function subtract(a: number, b: number): number {
    return a - b;
}
```

The `export` keyword makes `add()` and `subtract()` available to other files.

File: `main.ts`

```
import { add, subtract } from "./calculator";

console.log("Addition:", add(10, 20));
console.log("Subtraction:", subtract(20, 5));
```

Output

```
Addition: 30
Subtraction: 15
```

Modules: import and export in TypeScript

Here:

```
import { add, subtract } from "./calculator";
```

means:

- `import` → bring code into this file
- `{ add, subtract }` → functions we want to use
- `"./calculator"` → module/file from which they are imported

Example 2: Exporting a Class

File: `student.ts`

```
export class Student {
    name: string;
    rollNo: number;

    constructor(name: string, rollNo: number) {
        this.name = name;
        this.rollNo = rollNo;
    }

    display(): void {
        console.log("Name:", this.name);
        console.log("Roll No:", this.rollNo);
    }
}
```

File: `main.ts`

```
import { Student } from "./student";

let s1 = new Student("Rahul", 101);

s1.display();
```

Output

```
Name: Rahul
Roll No: 101
```

Example 3: Export Multiple Items

A single file can export multiple variables, functions, classes, etc.

File: `college.ts`

```
export const collegeName: string = "ABC Polytechnic";

export function displayCollege(): void {
    console.log("College:", collegeName);
}

export class Student {
    constructor(public name: string) {}

    display(): void {
        console.log("Student:", this.name);
    }
}
```

Now we can import all three.

File: `main.ts`

```
import { collegeName, displayCollege, Student } from "./college";

console.log(collegeName);

displayCollege();

let s1 = new Student("Amit");
s1.display();
```

Output

```
ABC Polytechnic
College: ABC Polytechnic
Student: Amit
```

Example 4: Import with an Alias

Sometimes we want to give an imported item a different name.

`calculator.ts`

```
export function add(a: number, b: number): number {
    return a + b;
}
```

`main.ts`

```
import { add as addition } from "./calculator";

console.log(addition(10, 20));
```

Output

```
30
```

Here:

```
add as addition
```

means we use the name `addition` in `main.ts`.

Example 5: Import Everything

We can import all exported members using `*`.

calculator.ts

```
export function add(a: number, b: number): number {
    return a + b;
}

export function multiply(a: number, b: number): number {
    return a * b;
}
```

main.ts

```
import * as Calculator from "./calculator";

console.log(Calculator.add(10, 20));
console.log(Calculator.multiply(5, 4));
```

Output

```
30
20
```

Here, `Calculator` acts as a namespace for the imported members.

Example 6: Default Export

A module can also have **one default export**.

student.ts

```
export default class Student {
    constructor(
        public name: string,
        public rollNo: number
    ) {}

    display(): void {
        console.log("Name:", this.name);
        console.log("Roll No:", this.rollNo);
    }
}
```

Notice:

Modules: import and export in TypeScript

```
export default
```

main.ts

```
import Student from "./student";

let s1 = new Student("Neha", 103);

s1.display();
```

Output

```
Name: Neha
Roll No: 103
```

With a default export, curly braces `{ }` are **not required**.

```
// Named export
import { Student } from "./student";

// Default export
import Student from "./student";
```

Named Export vs Default Export

Feature	Named Export	Default Export
Keyword	`export`	`export default`
Import syntax	`import { Student }`	`import Student`
Number per file	Multiple	One default
Curly braces	Required	Not required
Alias	`as` can be used	Import name can be chosen

Modules: import and export in TypeScript

Practical Example: Student Management

Suppose our project contains:

```
project/
│
├── student.ts
├── result.ts
└── main.ts
```

student.ts

```
export class Student {
    constructor(
        public name: string,
        public rollNo: number
    ) {}

    display(): void {
        console.log("Name:", this.name);
        console.log("Roll No:", this.rollNo);
    }
}
```

result.ts

```
export function calculateTotal(
    marks1: number,
    marks2: number,
    marks3: number
): number {
    return marks1 + marks2 + marks3;
}
```

main.ts

```
import { Student } from "./student";
import { calculateTotal } from "./result";

let student = new Student("Rahul", 101);

student.display();

let total = calculateTotal(80, 75, 90);

console.log("Total Marks:", total);
```

Output

```
Name: Rahul
Roll No: 101
Total Marks: 245
```

In one sentence:

`export` sends code from a module, while `import` receives and uses that code in another module.